

Astro Fight

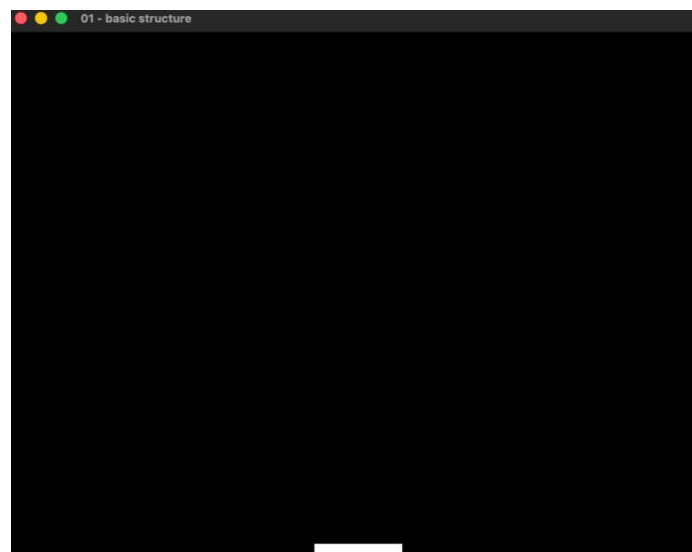
Malchiodi Riccardo s5680500

Tappa 01

In questa prima fase, l'obiettivo principale è disegnare su schermo la base della navicella(player). L'intero codice si basa sulla struttura iniziale del LAB3. Il codice introduce:

- **Variabili globali:** definiscono i parametri base immutabili, in particolare dimensioni della finestra a 800x600 e il max framerate a 60
- **Struct della navicella:** il costruttore inizializza automaticamente le dimensioni, impostate a 100.0 x 16.0 e calcola le coordinate per posizionare l'oggetto esattamente al centro in basso nello schermo
- **Struct state:** un contenitore logico per l'intero stato del gioco, al momento incapsula solo un'istanza di Spaceship andando a chiamare il suo metodo draw per visualizzarla.

Output ottenuto



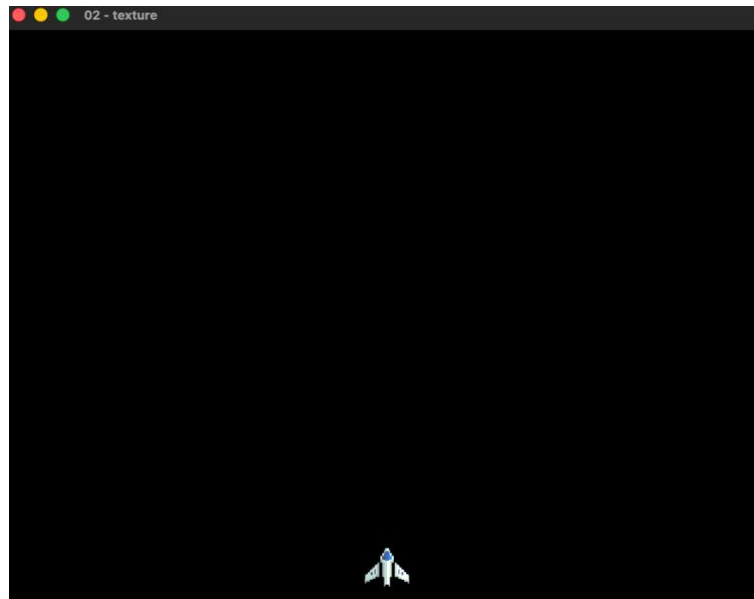
Tappa 02

In questa fase assegno una texture alla navicella. Per farlo ho creato un file .hpp che contiene l'immagine della texture trasformata in dati esadecimali. In questo modo il programma è completamente autocontenuto, perché non ha bisogno di leggere immagini da file esterni. La texture viene infatti caricata direttamente dal codice usando i dati salvati nell'array.

Per convertire l'immagine ho usato il sito :
<https://notisrac.github.io/FileToCArray/>

Asset usato : <https://foozlecc.itch.io/void-main-ship>

Output ottenuto



Tappa 03

Questa fase introduce la possibilità di muovere la navicella nelle quattro direzioni dello spazio dichiarando una costante di velocità e quattro metodi dedicati nella struct Spaceship.

```
void Spaceship::move_left(float elapsed)
{
    pos.x -= speed * elapsed;
}

void Spaceship::move_right(float elapsed)
{
    pos.x += speed * elapsed;
}

void Spaceship::move_up(float elapsed)
{
    pos.y -= speed * elapsed;
}

void Spaceship::move_down(float elapsed)
{
    pos.y += speed * elapsed;
}
```

elapsed rappresenta il tempo trascorso dall'ultimo aggiornamento, è molto importante per rendere il movimento indipendente dal framerate.

Nella struct State vengono aggiunte 4 variabili booleane inizializzate a false per registrare in tempo reale la pressione e il rilascio delle frecce direzionali.

Quindi uso un sistema **di Eventi a Callback**: l'idea è che abbiamo funzioni handle per intercettare gli eventi KeyPressed, KeyReleased e lo stato di focus della finestra. Ho anche inserito un template generico per catturare e ignorare qualsiasi evento non implementato, evitando errori di sistema.

Output ottenuto

Movimento della cella

Tappa 04

In questa fase implemento l'area di gioco. L'idea è quella di aggiungere due metodi (dentro a State):

1. **Field_limits**: si occupa di gestire i limiti la mia idea è quella di bloccare il giocatore sia a sinistra che a destra dello schermo, inoltre ho messo un limite (quindi un muro invisibile) più o meno a metà dello schermo in quanto l'area sopra a questo muro sarà dedicata ai nemici.

```
if(spaceship.pos.x < 0.0)
    spaceship.pos.x = 0.0;

if(spaceship.pos.x + spaceship.size.x > window_width)
    spaceship.pos.x = window_width - spaceship.size.x;

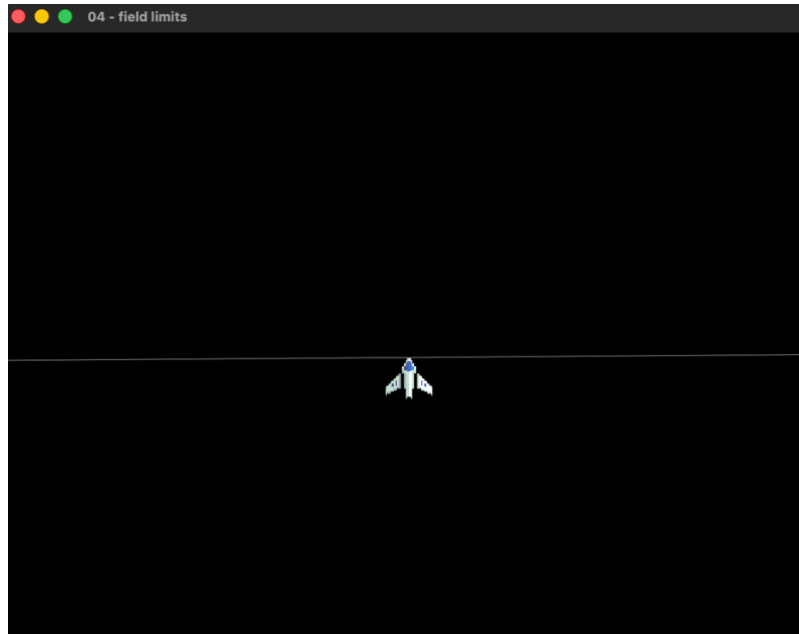
float limite_superiore = window_height / 2.0;

if(spaceship.pos.y < limite_superiore)
    spaceship.pos.y = limite_superiore;

if(spaceship.pos.y + spaceship.size.y > window_height)
    spaceship.pos.y = window_height - spaceship.size.y;
```

2. **Collisions**: al momento chiama soltanto field_limits e viene chiamato dentro all'update di state.

Output ottenuto



NOTA: nel gioco non esiste la linea centrale, l'ho inserita soltanto per far capire idealmente cosa succede.

Tappa 05

In questa fase introduco la possibilità per il giocatore di sparare premendo la barra spaziatrice. Ho creato una struct `Bullet`, la quale al momento memorizza solo una posizione. La cosa interessante è quella di avere una lista di proiettili dinamici (vector). In questa fase non ho applicato nessun tipo di texture al proiettile, dentro al metodo `draw` ho definito una **`RectangleShape`** e tramite un ciclo `for`, itero sul vector per spostare questa shape sulle coordinate di ciascun bullet.

```
sf::RectangleShape bShape(player_bullets_size);
bShape.setFillColor(sf::Color::Yellow);
for(size_t i = 0; i < bullets.size(); ++i)
{
    bShape.setPosition(bullets[i].pos);
    window.draw(bShape);
}
```

Problemi incontrati e soluzioni adottate

Il problema iniziale era capire che tipo di strutture dati usare per rappresentare i proiettili. Il mio dubbio principale era se usare una lista o array. Ho trovato molto utile questa spiegazione: <https://www.geeksforgeeks.org/dsa/difference-between-vector-and-list/>. L'idea è che il vector salva i dati in blocchi di memoria contigui, la lista invece salva tutto in RAM causando rallentamenti al gioco. Array statico invece il problema sarebbe stato di dover fissare una size.

Una volta scelta la struttura dati, il mio problema era quello di decidere COME implementare la logica dei proiettili. Siccome in SFML non possiamo creare

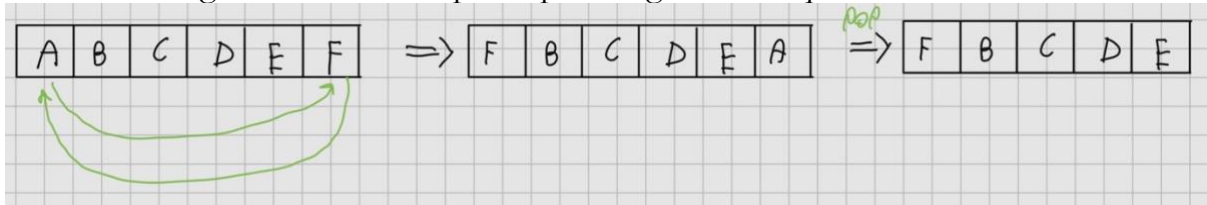
gameObject come in Unity (o Godot o addirittura UE) mi sono trovato in difficoltà. Facendo varie ricerche online ho trovato interessanti i seguenti forum:

- <https://stackoverflow.com/questions/26942267/list-of-bullets-c>
- <https://cplusplus.com/forum/general/67476/>

La mia soluzione definitiva è quella di usare l'algoritmo Swap and Pop trovato qui:

<https://forum.thegamecreators.com/thread/202835>

Lo schema logico che ho fatto per capire l'algoritmo è questo:



Come si vede dall'immagine non faccio nessun shift, se avessi usato il metodo erase (per esempio) il computer doveva eliminare A e poi spostare TUTTI gli altri di un posto e continuare col ciclo. In questo modo si riduce la complessità, infatti abbiamo $O(1)$. Questa è stata la conferma definitiva che usare Vector sia la scelta migliore nel mio gioco.

Ho creato dentro a **SpaceShip** un metodo fire, la sua implementazione è la seguente:

```
void Spaceship::fire()
{
    sf::Vector2f startPos = { pos.x + (size.x / 2.0f) -
                             (player_bullets_size.x / 2.0f), pos.y };
    bullets.push_back({startPos});
}
```

Dentro a **State::update** ho inserito l'algoritmo **swap and pop**:

```

for(size_t i = 0; i < spaceship.bullets.size(); )
{
    spaceship.bullets[i].pos.y -= player_bullets_speed * elapsed;

    if (spaceship.bullets[i].pos.y < -player_bullets_size.y)
    {
        std::swap(spaceship.bullets[i], spaceship.bullets.back());
        spaceship.bullets.pop_back();
    } else {
        ++i;
    }
}

```

NOTA: in questa fase ho notato una “problematica”, nel key pressed ho questa logica:

```

switch(key.scancode)
{
    case sf::Keyboard::Scancode::Left:
        state.move_spaceship_left = true;
        return;
    case sf::Keyboard::Scancode::Right:
        state.move_spaceship_right = true;
        return;
    case sf::Keyboard::Scancode::Up:
        state.move_spaceship_up = true;
        return;
    case sf::Keyboard::Scancode::Down:
        state.move_spaceship_down = true;
        return;
    case sf::Keyboard::Scancode::Space:
        state.spaceship.fire();
        return;
    default:
        return;
}

```

Mi sono accorto in fasi successive (alla fase 9) che il comportamento non è quello voluto, in quanto lasciando il codice così il giocatore se tiene premuto il tasto spazio spara all’infinito, la mia idea di gameplay è che il giocatore per sparare deve premere di fila la barra spaziatrice. Per risolvere basta fare questo:

```

case sf::Keyboard::Scancode::Space:
    if (state.spaceship.fire_timer >= state.spaceship.fire_rate)
    {
        state.spaceship.fire();
        state.spaceship.fire_timer = 0.0f;
    }

```

Tappa 06

In questa fase implemento il primo nemico, inizialmente statico e senza texture.

La cosa più interessante di questa fase sono le hitbox, sia dei proiettili e quella del nemico.

Ho dovuto modificare il metodo **State::Collision**. Nelle fasi precedenti si dedicava solamente di controllare il **field limits**. Quello che deve fare ora è controllare se i proiettili si intersecano con la figura, in caso affermativo allora distrugge la shape (e anche i proiettili che hanno faccio collision con essa).

Problemi incontrati e soluzioni adottate

Il problema principale è capire come funzionano le collisioni in SFML, come nella fase precedente il mio problema è stato capire come trasformare la logica da Unity a SFML, siccome non abbiamo un boxCollider , dobbiamo creare tutto noi manualmente.

ho seguito questo video:

https://www.youtube.com/watch?v=AWDYJma5bmE&list=PL6xSOsbVA1eaJnHo_O6uB4qU8LZWzzKdo&index=10

Essendo però un video di 8 anni fa la versione SFML è diversa dalla mia attuale.

Ho consultato questa pagina:

https://www.sfm-dev.org/documentation/3.0.0/classsf_1_1Rect.html .

Inoltre nel video viene usato **.Intersects()** ma ormai è deprecato, ho usato la versione nuova **.findIntersection()**.

```
template<typename T>
std::optional< Rect< T > > sf::Rect< T >::findIntersection (const Rect< T > &rectangle) const
```

Check the intersection between two rectangles.

Parameters

rectangle Rectangle to test

Returns

Intersection rectangle if intersecting, std::nullopt otherwise

See also

contains

All'interno di State::collisions:

```

for (size_t i = 0; i < spaceship.bullets.size(); )
{
    sf::FloatRect bulletRect(spaceship.bullets[i].pos, player_bullets_size);
    sf::FloatRect enemyRect(enemy.pos, enemy.size);

    if (bulletRect.findIntersection(enemyRect))
    {
        enemy.isAlive = false;

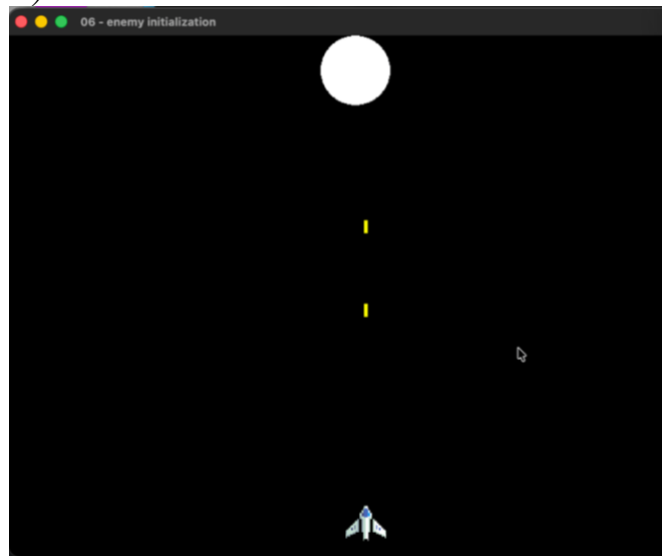
        std::swap(spaceship.bullets[i], spaceship.bullets.back());
        spaceship.bullets.pop_back();
    }
    else
    {
        i++;
    }
}

```

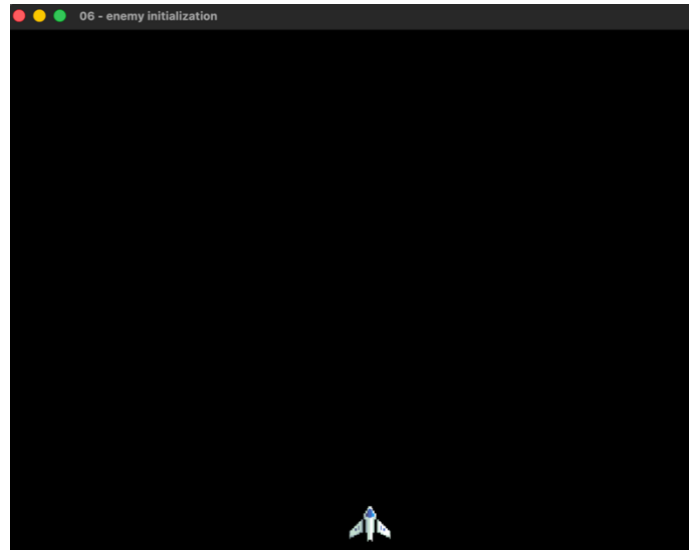
sf::FloatRect crea un rettangolo virtuale attorno a un oggetto, quindi simula abbastanza il concetto di boxCollider di cui parlavo prima.

Output ottenuto

Prima(il giocatore spara):



Dopo(shape distrutta):



Tappa 07

In questa fase comincio a fare level design. Assegno la texture al nemico come ho fatto col player nella fase 2. Le modifiche rilevanti sono l'immagine di sfondo ed estetica dei proiettili.

La struct Bullet viene arricchita con **anim_timer** e **current_frame**. Tramite l'operatore modulo (% 3), i frame cicllano in un loop infinito (0 -> 1 -> 2 -> 0) ogni 0.1 secondi (definito da swap_interval)

```
// bullet animation
spaceship.bullets[i].anim_timer += elapsed;

if (spaceship.bullets[i].anim_timer > swap_interval)
{
    spaceship.bullets[i].anim_timer = 0.0; // timer reset
    spaceship.bullets[i].current_frame = (spaceship.bullets[i].current_frame + 1) % 3;
}
```

Problemi incontrati e soluzioni adottate

Problema 1: Il problema attuale è che la hitbox del proiettile è 4x15 ma ora ho un'immagine 64x64. Quindi devo aggiustare l'asse X e Y. Per farlo uso questa formula:

Asse X: faccio $64 - 4 / 2 = 30$, quindi “arretriamo” il disegno a sinistra di 30 pixel per centrarlo.

Asse Y: Stesso ragionamento e quindi $64 - 15 / 2 = 24.5$

Codice di Spaceship::draw

```

sf::Sprite bSprite(bullet_texture);

bSprite.setScale({2.0f, 2.0f});

for(size_t i = 0; i < bullets.size(); ++i)
{
    bSprite.setTextureRect(sf::IntRect({bullets[i].current_frame * 32, 0},
                                       {32, 32}));

    float offset_x = bullets[i].pos.x - 30.0f; // (64 - 4) / 2
    float offset_y = bullets[i].pos.y - 24.5f; // (64 - 15) / 2

    bSprite.setPosition({offset_x, offset_y});
    window.draw(bSprite);
}

```

sf::IntRect serve a definire una porzione della texture in coordinate pixel. Con **setTextureRect** io applico questa porzione allo sprite, cioè dico a SFML di disegnare solo quella parte dell'immagine invece della texture intera. Questa logica l'ho presa dal file caricato su aulaweb **main_animated_sprite_smooth.cpp**.

Problema 2: la mia idea iniziale era quello di avere uno sfondo animato ma avrebbe portato a tantissime righe di codice nel file hpp (50.000+). Mi sono ispirato a questo video <https://www.youtube.com/watch?v=1TJgqKXA88> Quindi la mia soluzione definitiva è stata quella di avere uno sfondo statico, ho usato questa immagine: <https://enjl.itch.io/background-starry-space>.

Siccome ho settato la window ad una dimensione di 800 x 600 allora ho pensato di aggiungere al costruttore di background un rettangolo, in questo modo vado a “colorare” il rettangolo applicandoli la texture.

```

Background::Background()
{
    texture = sf::Texture(bg_png, bg_png_len);
    texture.setRepeated(true);
    shape.setSize({800.0f, 600.0f});
    shape.setTexture(&texture);
    shape.setTextureRect(sf::IntRect({0, 0}, {800, 600}));
}

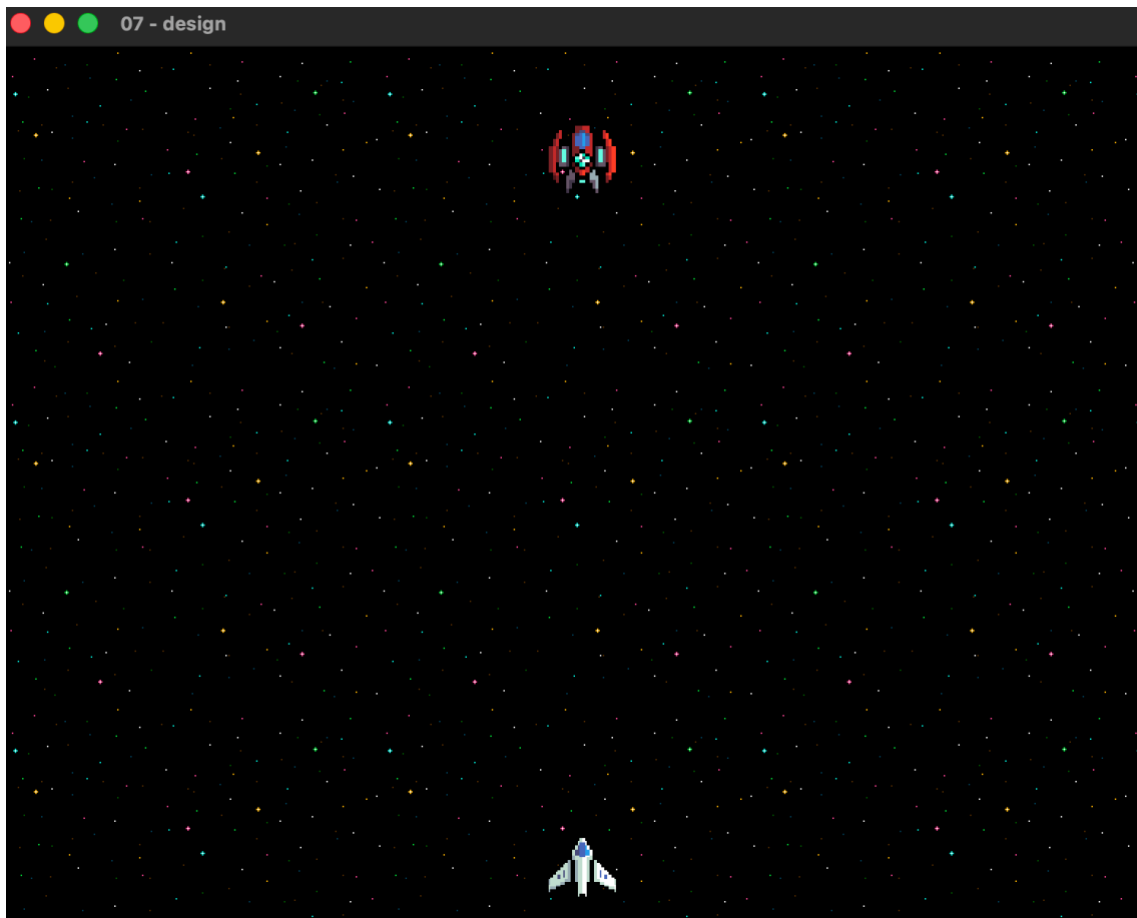
```

Ho usato **setRepeated** per riempire dei “buchi”, l'immagine sfondo è molto più piccola di 800x600 quindi mi permette di “looparla.”

Ho trovato questo suggerimento qua:

<https://stackoverflow.com/questions/26517066/repeating-texture-to-fit-certain-size-in-sfml>

Output ottenuto



Tappa 08

In questa fase il nemico si muove e spara al giocatore.

Alla struct **Enemy** vengono aggiunte le variabili **speed** e **moving_right**.

Nell'`update(state)`, il nemico si muove orizzontalmente e inverte automaticamente la direzione non appena le sue coordinate collidono con i bordi sinistro o destro dello schermo.

```
if (enemy.moving_right) {
    enemy.pos.x += enemy.speed * elapsed;
} else {
    enemy.pos.x -= enemy.speed * elapsed;
}

if (enemy.pos.x + enemy.size.x >= window_width)
{
    enemy.moving_right = false;
}

if (enemy.pos.x <= 0)
{
    enemy.moving_right = true;
}
```

Analogamente al giocatore, il nemico possiede una lista di proiettili e una texture dedicata, ma in più gestisce un `fire_timer` per avere un tempo di ricarica (cooldown) tra un colpo e l'altro.

Ho creato il metodo **State::restart()** in questo modo:

```
void State::restart()
{
    spaceship = Spaceship();
    enemy = Enemy();
    // player movements reset
    move_spaceship_left = false;
    move_spaceship_right = false;
    move_spaceship_up = false;
    move_spaceship_down = false;
}
```

Problemi incontrati e soluzioni adottate

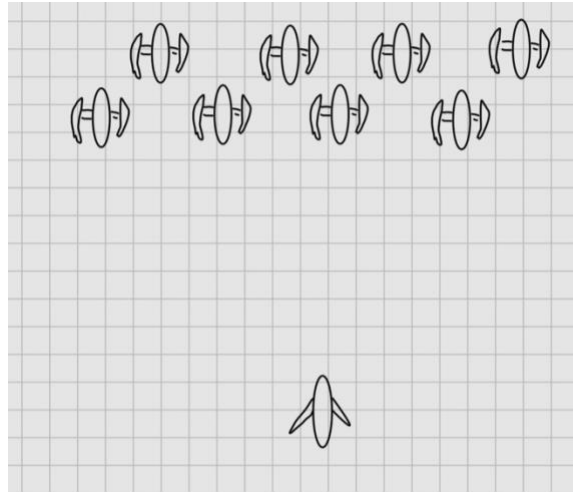
Problema (Orientamento e Origine dei Proiettili): Usando la stessa texture e logica del giocatore, i proiettili nemici non partivano dal punto giusto ed erano orientati verso l'alto.

Soluzione: Nel metodo **fire()** del nemico ho aggiunto `pos.y + size.y` all'asse Y per far partire i bullets dal "fondo" dell'astronave nemica. Inoltre, nel metodo `draw`, lo sprite del proiettile è stato ruotato fisicamente verso il basso:

```
bSprite.setRotation(sf::degrees(180.0));
```

Tappa 09

Questa è la fase che rappresenta il gameplay definitivo. In questa fase ho implementato un sistema ad ondate. Questa volta siccome ho deciso che voglio implementare 8 nemici per ondata per un totale di 8 waves ho usato un array statico. Questo array l'ho inserito nello State. I nemici si muovono tutti insieme in un blocco sincronizzato. Quando un qualsiasi nemico tocca il bordo della finestra, l'intero blocco di nemici cambia direzione e scende di 20 pixel (come in `spaceInvaders`).



Problemi incontrati e soluzioni adottate

Il problema principale di questa fase è stato come implementare il sistema di wave, mi sono fatto aiutare da **Gemini Pro 3.1**.

Quello che mi è stato suggerito è quello di calcolare dinamicamente una griglia basata su righe e colonne. Utilizzando l'operatore modulo (%) e la divisione intera (/), ho potuto assegnare a ciascun nemico la corretta coordinata X e Y di partenza all'interno del ciclo di spawn.

```
for(int i = 0; i < enemies_per_wave; ++i)
{
    int col = i % enemy_cols;
    int row = i / enemy_cols;

    enemies[i] = Enemy();
    enemies[i].pos.x = start_x + col * enemy_spacing_x;
    enemies[i].pos.y = start_y + row * enemy_spacing_y;
    enemies[i].fire_timer = (float) i * (enemy_fire_rate / enemies_per_wave);
}
```

Come si nota nell'ultima riga del codice soprastante, per evitare che tutti gli 8 nemici sparassero il loro primo colpo ho moltiplicato il loro indice i per una frazione del fire rate.

La logica di discesa è la seguente:

```

float drop_amount = 20.0f;
if (max_y + drop_amount > limite_inferiore)
{
    drop_amount = limite_inferiore - max_y;
    if (drop_amount < 0.0f)
    {
        drop_amount = 0.0f;
    }
}

for (int i = 0; i < enemies_per_wave; ++i)
{
    if(!enemies[i].isAlive) continue;

    enemies[i].pos.y += drop_amount;
}

```

Tappa 10

In questa fase aggiungo sfx e musica di sottofondo.

Assets usati: <https://murkje.itch.io/retro-sfx>

bg: <https://downloads.khinsider.com/game-soundtracks/album/star-soldier-nes/03%2520Main%2520Theme.mp3>

Per l'implementazione audio ho seguito la documentazione ufficiale di SFML : <https://www.sfm1-dev.org/tutorials/3.0/audio/sounds/#loading-and-playing-a-sound> e anche questo video Youtube: <https://www.youtube.com/watch?v=1OBzL28jFPk>

Per il background ho usato un file .ogg in quanto facendo ricerche online (lo dice anche nel video Youtube) ho scoperto che questo formato offre una buona qualità audio mantenendo dimensioni ridotte grazie alla compressione, permettendo quindi di risparmiare spazio su disco. Inoltre è abbastanza lo standard nei videogiochi. Fonti che mi sono tornati utili:

- https://www.reddit.com/r/gamedev/comments/1gtmxoc/what_format_do_you_use_for_music_in_your_game_wav/
- <https://www.youtube.com/watch?v=UKzJURlCT0E&t=51s>
- <https://www.quora.com/Why-do-video-games-use-the-ogg-file-format-for-sounds-so-much>

Per gli effetti sonori vanno bene i file .wav in quanto vengono pre-caricati nella RAM tramite i buffer all'avvio del gioco, garantendo una riproduzione istantanea senza latenza. Le risorse audio sono state inserite nella struct State perché sono collegate agli eventi di gioco (spari, bg e collisioni) e quindi mantenerla nello stesso contenitore ha senso per pulizia di codice e ordine.

```
// bg music
sf::Music bgMusic;

// sound buffer
sf::SoundBuffer shootBuffer;
sf::SoundBuffer enemyShootBuffer;
sf::SoundBuffer hitBuffer;

// sound to add to the sound buffer
sf::Sound shootSound;
sf::Sound enemyShootSound;
sf::Sound hitSound;
```

Il buffer rappresenta la risorsa audio in memoria, mentre `sf::Sound` è l'istanza che la riproduce. Questa separazione è fondamentale per evitare ricaricamenti continui e per gestire più effetti sonori simultanei in un sistema real-time. `Music` invece non usa buffer, è uno stream diretto da disco.

Tappa 11

In questa fase aggiungo le esplosioni ai nemici, lo faccio andando a dichiarare una struct nuova chiamata **Explosion**. Nello State devo aggiungere un vettore che mi permetta di individuare le posizioni delle esplosioni e la texture da applicargli. Questo mi è stato suggerito da **Gemini 3.1 Pro**, l'idea sarebbe che ogni volta che il nemico viene colpito dal bullet rimane "bloccato" nella posizione di collisione e faccia partire l'animazione associata. Quindi ogni volta che il proiettile del **Player** colpisce il **Nemico** faccio una **push_back** in questo vettore. La cosa interessante che a differenza dei nemici, andiamo a eliminare l'animazione, questo perché tenerle "spente" sprecherebbe ulteriore memoria. Siccome ho già deciso di non eliminare i nemici, non eliminare neanche le animazioni avrei appesantito ulteriormente il codice.

```

for (size_t i = 0; i < explosions.size(); )
{
    explosions[i].anim_timer += elapsed;
    if (explosions[i].anim_timer > 0.05)
    {
        explosions[i].anim_timer = 0.0;
        explosions[i].current_frame++;
    }

    if (explosions[i].current_frame > 9)
    {
        std::swap(explosions[i], explosions.back());
        explosions.pop_back();
    }
    else
    {
        i++;
    }
}

```

Uso un timer per controllare la velocità dell'animazione. Ogni 0.05 secondi avanzo di un frame dello spritesheet. Quando arrivo all'ultimo frame, considero l'esplosione conclusa e la rimuovo dal vettore.

Tappa 12

In questa fase vado ad implementare la finestra di win e di gameOver.

Per la scritta ho seguito lo stile dei codici riportati su aula web, quindi importare un font.ttf e nel main vado a preparare tre oggetti **sf::Text**: "GAME OVER" (colore rosso), "YOU WON!" (colore verde) e "Press R to Restart" (colore bianco).

La struct State viene arricchita con due flag booleani inizializzati a false (gameOver e win). Quando la navicella viene colpita da un proiettile nemico gameOver diventa vero, mentre se le ondate giungono al termine win diventa vero.

La funzione handle relativa alla pressione dei tasti viene aggiornata. Se ci si trova nella schermata di fine partita, il gioco smette di accettare comandi di movimento o sparo e resta in attesa esclusiva del tasto R per chiamare la funzione restart(). Ho deciso inoltre di aggiungere dei suoni (wav) per la schermata di vittoria e gameOver.

Problemi incontrati e soluzioni adottate

Il problema principale di questa fase era capire come "congelare" il gioco al momento della morte o della vittoria, facendo sparire gli elementi di gioco per mostrare solo il testo.

La prima idea che ho avuto è quella di mettere la seguente if:

```

if(gameOver || win)
{
    return;
}

```


Dentro a State::update, questo impedisce istantaneamente qualsiasi calcolo di movimento, collisione o animazione.

L'ultima cosa che ho fatto è inserire nel main questo blocco di codice:

```
if (!state.gameOver && !state.win)
{
    state.draw (window);
} else if(state.gameOver){
    window.draw(gameOverText);
    window.draw(restartText);
} else if(state.win){
    window.draw(winText);
    window.draw(restartText);
}
```

Il metodo state.draw viene chiamato solo se il gioco è ancora in corso. Se scatta la fine della partita, ignoro lo State e disegno unicamente gli oggetti di testo.

Output ottenuto





Tappa 13

In questa fase aggiungo un counter dei nemici e score del giocatore. Ho aggiunto nella struct State una nuova variabile score inizializzata a 0. All'interno del metodo `State::collisions()`, ogni volta che un proiettile colpisce un nemico, il punteggio viene incrementato di 100 punti. Il punteggio viene poi azzerato in caso di restart. Nel Game Loop del main vengono istanziati due nuovi testi `sf::Text`: uno per lo "SCORE" e uno per gli "ENEMIES" (nemici rimanenti), posizionati in alto a destra e al centro dello schermo. Per il conteggio dei nemici rimanenti non considero solo i nemici della wave attuale, ma anche quelli delle wave successive. Il valore viene calcolato moltiplicando le wave ancora da affrontare per il numero di nemici per wave e sommando i nemici ancora vivi nella wave corrente. In questo modo il giocatore ha sempre una visione del progresso complessivo della partita.

Problemi incontrati e soluzioni adottate

Avrei potuto usare un semplice contatore inizializzato a 64 e decrementarlo a ogni uccisione. Ho preferito però ricalcolare il numero dei nemici rimanenti a partire dallo stato reale del gioco, usando wave corrente e nemici attivi. Se avessi fatto un

semplice `remaining_enemies` avrei dovuto stare attento a restart, vittoria, cambio wave ecc.

```
int total_remaining = (total_waves - state.current_wave - 1) * enemies_per_wave + state.active_enemies;
```

Esempio: se sono alla wave 3 (`current_wave = 2`) e in questa wave ci sono 5 nemici in vita avrei: $(8 - 2 - 1) * 5 * 8 + 5 = 45$ nemici in totale.

Output ottenuto



Tappa 14

Questa è la fase finale. in questa fase ho deciso di modificare la struct `Enemy` per renderla più leggera. Il problema fino a questa tappa era che caricavo per i nemici due texture ogni volta, siccome abbiamo 8 nemici per un totale di 64 nemici, stavo caricando 8 copie dell'immagine del nemico e 8 copie dell'immagine del proiettile, la cosa migliore da fare è metterlo dentro a state come ho fatto con le esplosioni, in questo modo il computer salva in memoria solo una copia per il nemico e una per il proiettile risparmiando memoria. Per il giocatore posso lasciarla come ho fatto fino adesso in quanto stiamo parlando di un solo spazio in RAM occupata e quindi inserirlo nello state a livello di prestazione non avrebbe cambiato nulla.